

Projet F1

Guide de progression : comprendre la chaîne, durcir le code, préparer les extensions

Ce document n'est pas la correction du projet. C'est un manuel de notions appliquées : chaque chapitre part d'un mécanisme que le projet utilise vraiment, explique en quoi il est piégeux, puis propose des exercices à faire dans le code réel.

Méthode de lecture : un chapitre, puis le fichier concerné ouvert à côté, puis l'exercice, puis les tests du maillon. Un guide lu passivement ne rend personne plus fort ; c'est le passage à l'écriture qui fait la différence.

Sommaire

Chapitre	Contenu	À lire avec
1. La chaîne et les contrats	Découplage, formats figés, sentinelles	ENONCE.md
2. Python	Parsing, flottants, encodage, données sales	01-python/
3. Java	Records, agrégation, comparateurs, exceptions	02-java/
4. JavaScript	Tri, DOM, événements, état d'affichage	03-js/
5. Méthode	Débogage, tests, Git propre, revue	tout le dépôt
6. Extensions	E1 à E4 : objectifs et pièges	extensions/
7. Aide-mémoire	Extraits prêts à adapter	—
8. Feuille de route	État réel, prochaines actions	README.md

1. La chaîne et les contrats

Le projet fait circuler la même information dans trois langages. Chaque passage de relais est un fichier au format figé : c'est un contrat de données. Tant que le contrat tient, chaque maillon peut être réécrit librement ; dès qu'il change, les trois maillons cassent en même temps.

```
donnees/resultats.csv      brut, 6 colonnes, casse libre
|
v Python : lire, nettoyer, normaliser
02-java/courses_propres.csv  CONTRAT 1 : 5 colonnes figees
|
v Java : agréger, trier, calculer
03-js/donnees.js           CONTRAT 2 : PILOTES et ECURIES
|
v JavaScript : trier, filtrer, afficher
page de classements
```

1.1 Pourquoi trois langages

Maillon	Rôle réel	Ce qu'il apporte
Python	ETL : extraction, transformation, chargement	Itération rapide, manipulation de texte, scripts de contrôle
Java	Moteur typé : agrégation, tri, calcul de règles	Types, collections, testabilité, performances prévisibles
JavaScript	Rendu : interaction et affichage	Accès direct au DOM, événements, mise à jour sans rechargement

Ce découpage imite un projet réel : chaque maillon est choisi pour ce qu'il fait le mieux, et le fichier intermédiaire remplace les appels directs entre programmes. Le prix à payer, c'est la discipline sur le format.

1.2 Le contrat 1 — courses_propres.csv

```
course;pilote;ecurie;position;temps_tour
Bahrein;VERSTAPPEN;Red Bull;1;93.614
Bahrein;HAMILTON;Mercedes;0;
```

- Séparateur point-virgule, une ligne d'en-tête, ordre des colonnes figé.
- Un abandon : position 0 et temps vide (pas la chaîne None, pas un 0).
- Temps en secondes, trois décimales, UTF-8 sans BOM, fin de ligne LF.
- Pas de colonne statut : l'information est encodée dans la position.

1.3 Le contrat 2 — donnees.js

- Deux constantes globales : PILOTES et ECURIES.
- Tableaux d'objets aux mêmes noms de champs : nom, ecurie, points, victoires.
- C'est du JavaScript, pas du JSON strict : un fichier JS accepte des commentaires, un JSON non.

1.4 Les sentinelles : utiles, mais à confiner

Couche	abandon	temps absent	ecurie absente
CSV contrat 1	position 0	champ vide	(pas de cas)
Python	position 0	None	chaîne vide
Java	position 0	tempsTour -1	chaîne vide
JavaScript	position 0	jamais	chaîne vide

Le danger n'est pas la sentinelle, c'est son transport. None écrit tel quel produit la chaîne "None" dans un CSV. -1 oublié en Java devient un temps réel. Une position 0 comptée dans une moyenne fausse le résultat. Règle pratique : sentinelle définie à la frontière, convertie tout de suite, jamais transportée plus loin que nécessaire.

Règle d'or

1. On fige le contrat avant d'écrire du code.
2. On convertit les sentinelles à l'entrée et à la sortie, jamais au milieu.
3. Un seul format entre deux langages, et il est documenté.

Exercice 1.1 — lire un contrat comme une spécification

- Pour chaque colonne du contrat 1, écrivez : type, plage de valeurs, valeur qui signifie « rien ».
- Comptez ensuite les fonctions qui connaissent la sentinelle. Plus de deux, c'est une fuite à corriger.
- Provoquez le bug : écrivez None dans une colonne et regardez le fichier produit.

2. Python — lire, convertir, écrire sans se mentir

Ce maillon paraît simple parce que le fichier est petit. En réalité il concentre trois sujets classiques : le parsing d'un format texte, la représentation des nombres, et la frontière entre données valides et données douteuses.

2.1 Convertir un chronomètre : pourquoi pas float() ?

```
>>> float("1:33.614")
ValueError: could not convert string to float: '1:33.614'
```

Un chronomètre n'est pas un nombre, c'est un texte structuré. La conversion suit donc deux étapes : découper, puis recomposer.

```
def temps_en_secondes(texte):
    texte = texte.strip()
    if not texte:                # champ vide = pas de temps
        return None
    minutes, secondes = texte.split(":") # 2 morceaux attendus
    return round(int(minutes) * 60 + float(secondes), 3)
```

- `minutes, secondes = ...` n'est sûr que parce que le format est garanti par l'énoncé. Un troisième deux-points fait lever `ValueError: too many values to unpack`.
- `int(minutes)` documente l'intention (les minutes sont entières) et refuse plus tôt une donnée aberrante.
- `texte.strip()` d'abord : sans lui, un espace en trop transforme un nombre valide en exception.

Variante défensive, utile dès que la source devient douteuse (extension E1) :

```
import re
MOTIF = re.compile(r"^(\d+):(\d{2})\.(\d{3})$")

def temps_en_secondes(texte):
    correspondance = MOTIF.match(texte.strip())
    if correspondance is None:
        return None
    minutes, secondes, millisecondes = correspondance.groups()
    return int(minutes) * 60 + int(secondes) + int(millisecondes) / 1000
```

Le motif valide le format **et** capture les morceaux. Le coût est une ligne supplémentaire à comprendre ; le gain est qu'aucune entrée ne peut produire une valeur silencieusement fausse.

Piège classique

Un `try / except ValueError: return None` autour de tout le corps attrape aussi vos propres bugs (variable mal nommée, indice faux) et les transforme en « temps absent ». Pendant le développement, laissez l'exception remonter : elle dit le fichier et la ligne.

2.2 Nombres flottants : ce que Python ne peut pas représenter

```
>>> 0.1 + 0.2 == 0.3
False
>>> round(2.675, 2)
2.67          # 2.675 est stocké 2.67499999...
```

Un `float` est un binaire à 53 bits de mantisse. La conséquence pratique pour ce projet : un temps « rond » n'existe pas forcément, et deux calculs équivalents sur le papier peuvent différer d'un bit.

- `round(x, 3)` arrondit la valeur ; `f"{x:.3f}"` fixe l'écriture. Le contrat porte sur l'écriture : c'est le second qui compte pour les tests.

- `str(93.6)` donne "93.6", alors que le contrat exige 93.600. Trois décimales, toujours.
- Pour comparer deux temps relus d'un fichier, ne faites pas `==` : utilisez `math.isclose(a, b, abs_tol=1e-9)`.

2.2.1 Le réflexe qui supprime le problème : les millisecondes

```
temps_ms = int(minutes) * 60_000 + int(secondes) * 1_000 + int(ms)
           # 93614 millisecondes au lieu de 93.614 secondes

def ecrire_secondes(temps_ms):
    return f"{temps_ms // 1000}.{temps_ms % 1000:03d}"
```

Quand une grandeur a une précision décimale exacte (un chronomètre, un montant en centimes), la représenter par un entier évite toute la discussion sur les flottants : les additions restent exactes et le formatage n'est qu'une mise en forme finale. C'est le choix fait par la plupart des systèmes de chronométrage.

2.3 L'ordre des instructions change la solidité du code

Deux versions de la même lecture, dont une seule survit à une ligne d'abandon. La différence n'est pas le style, c'est la présence de `int("")` sur le chemin critique.

```
# fragile : int("") explose avant meme d'avoir lu le statut
position = int(colonnes[3])
statut = colonnes[5]
if statut == "ABANDON":
    position = 0

# solide : on lit le statut d'abord, on convertit ensuite
statut = colonnes[5]
if statut == "ABANDON":
    position = 0
else:
    position = int(colonnes[3])
```

- Principe général : convertir le plus tard possible, quand toutes les conditions qui peuvent éviter la conversion sont connues.
- `int("")` et `float("")` lèvent une `ValueError` ; aucun des deux ne renvoie 0.

2.4 La sentinelle qui fuit : None écrit dans un fichier

```
>>> str(None)
'None'

temps = ligne["temps_tour"]
champ = "" if temps is None else f"{temps:.3f}" # correct
champ = f"{temps:.3f}"                        # plante sur None
```

C'est le bug le plus coûteux du maillon : le fichier reste lisible, les tests Java échouent plus loin, et on cherche l'erreur dans le mauvais langage. Le réflexe est de convertir la sentinelle à la sortie, dans la seule fonction qui écrit.

2.5 Encodage, BOM, fins de ligne : les trois pièges du fichier

```
# lecture : tolere un BOM eventuel (export Excel / Windows)
with open(chemin, "r", encoding="utf-8-sig") as fichier:
    lignes = fichier.read().splitlines()

# ecriture : LF uniquement, comme la reference du projet
with open(chemin, "w", encoding="utf-8", newline="\n") as fichier:
    fichier.write("course;pilote;ecurie;position;temps_tour\n")
```

- Sans `encoding=`, Python utilise la locale du système (souvent cp1252 sous Windows) : « PÉREZ » devient illisible et les accents cassent silencieusement.
- Le BOM se lit avec `utf-8-sig` mais ne s'écrit jamais : sinon la première colonne devient `\ufeffcourse` et l'en-tête du contrat ne correspond plus.
- `csv.writer` écrit du `\r\n` par défaut sous Windows : pratique pour Excel, fatal pour un contrat comparé ligne à ligne. Ici on écrit soi-même, ou on ouvre avec `newline=""`.

2.6 Lire un fichier sale sans se laisser surprendre

```
with open(chemin, encoding="utf-8-sig") as fichier:
    fichier.readline()          # en-tête
    for numero, brute in enumerate(fichier, start=2):
        brute = brute.rstrip("\n")
        if not brute.strip():   # ligne vide en fin de fichier
            continue
        colonnes = brute.split(";")
        if len(colonnes) != 6:
            raise ValueError(
                f"ligne {numero} : {len(colonnes)} colonnes au lieu de 6")
```

- `enumerate(..., start=2)` donne le numéro de ligne du fichier : un message d'erreur qui dit « ligne 37 » fait gagner une heure.
- Valider le nombre de colonnes avant d'indexer évite `IndexError` et surtout évite de lire la mauvaise colonne sans rien remarquer.
- **Asymétrie à connaître** : en Python, `"a;b;;".split(";")` renvoie 4 éléments, champs vides finaux compris. En Java, `"a;b;;".split(";")` n'en renvoie que 2 : les vides finaux sont supprimés. Sur un contrat qui finit par un temps vide, la différence fait planter le maillon suivant.
- Et `split()` sans argument ne découpe pas sur le point-virgule : il découpe sur les espaces. `"a;b".split()` renvoie `["a;b"]`.

2.7 Écrire des tests qui apprennent quelque chose

La cellule de tests fournie vérifie que le code marche. Un bon test supplémentaire vérifie une **propriété** : quelque chose qui doit rester vrai quelles que soient les données.

- Propriété de bord : un champ vide renvoie `None`, jamais 0, jamais `""`.
- Propriété d'aller-retour : écrire puis relire doit redonner exactement les mêmes valeurs (mêmes types compris).
- Propriété d'idempotence : nettoyer deux fois donne le même résultat que nettoyer une fois.

```
# tests/test_ingestion.py (pytest)
import importlib
import pytest

ingestion = importlib.import_module("ingestion") # cf. 2.9

@pytest.mark.parametrize("texte, attendu", [
    ("1:33.614", 93.614),
    ("0:59.500", 59.5),
    ("", None),
    (" ", None),
])
def test_temps_en_secondes(texte, attendu):
    assert ingestion.temps_en_secondes(texte) == attendu

def test_ecriture_ne_produit_jamais_le_mot_None():
    contenu = produire_le_contrat()
    assert "None" not in contenu
```

```
pytest -q # tout
pytest -q -k temps # les tests dont le nom contient temps
pytest -q --maxfail=1 # s'arrete au premier echec
pytest -q -x --pdb # ouvre le debogueur sur l'echec
```

Le paramétrage remplace des dizaines de fonctions identiques, et le nom du paramètre apparaît dans le rapport d'échec : c'est gratuit et ça accélère le diagnostic.

2.8 Extension E1 — nettoyer des données sales

Le fichier de E1 est le même export, mais tel qu'il sort du système : casse incohérente, accents, espaces parasites, virgule décimale, doublons, écuries inconnues. Ce n'est pas un exercice de style : c'est le quotidien de tout pipeline de données.

```
Ordre des etapes : espaces -> casse/accents -> ecuries ->
                  separateur decimal -> validation -> doublons

import unicodedata

def sans_accents(texte):
    decompose = unicodedata.normalize("NFD", texte)
    return "".join(c for c in decompose
                    if unicodedata.category(c) != "Mn")

def normaliser_pilote(texte):
    return sans_accents(texte).strip().upper() # "Pérez" -> "PEREZ"
```

- NFD sépare la lettre de son accent ; le filtre Mn (non-spacing mark) supprime l'accent. C'est la méthode fiable, contrairement à une liste de replace qui oublie toujours un caractère.
- Pour comparer du texte en ignorant la casse, casefold() est plus complet que lower() (il gère aussi les alphabets non latins). Ici upper() suffit car le contrat exige des majuscules.
- Les écuries ne se normalisent pas, elles se **rapprochent d'un dictionnaire connu** : "MCLAREN".title() donne "McLaren", ce qui ne correspond à aucune écurie du championnat.

```
ECURIES_CONNUES = {
    "RED BULL": "Red Bull", "FERRARI": "Ferrari",
    "MERCEDES": "Mercedes", "MCLAREN": "McLaren",
    "ASTON MARTIN": "Aston Martin",
}

def normaliser_ecurie(texte):
    cle = sans_accents(texte).strip().upper()
    return ECURIES_CONNUES.get(cle) # None si inconnue
```

```
vues = set() # dedoublonnage en conservant l'ordre
propre = []
rejetees = 0
for ligne in lignes_sales:
    cle = (ligne["course"], ligne["pilote"], ligne["position"],
          ligne["temps_tour"])
    if cle in vues:
        continue
    vues.add(cle)
    propre.append(ligne)
```

- set ne conserve pas l'ordre : on l'utilise comme mémoire des clés vues, et la liste garde l'ordre d'origine. Un `list(set(...))` mélangerait les lignes du contrat.
- " RED BULL " avec espaces internes multiples : " ".`join(texte.split())` normalise en une seule passe.

Choisir entre tolérer et refuser

Dans un import réel, on **tolère** ce qu'on comprend et on journalise ce qu'on jette : le pipeline ne s'arrête pas pour une ligne abîmée.

Dans un test ou une vérification de contrat, on **refuse** : une anomalie doit faire échouer, sinon elle se propage silencieusement.

Le bon réflexe est de compter et d'afficher : « 50 lignes lues, 3 rejetées, 2 doublons ». Un pipeline muet n'est pas un pipeline fiable.

2.9 Du notebook au script : rendre le maillon automatisable

Un notebook est excellent pour explorer et discuter, mauvais pour être enchaîné dans un pipeline. La conversion est mécanique.

```
cd 01-python
jupyter nbconvert --to script ingestion.ipynb # produit ingestion.py
```

- Le fichier produit contient tout, y compris la cellule de tests : déplacez les tests dans `tests/` et gardez le script opérationnel.
- Ajoutez `if __name__ == "__main__":` : importer vos fonctions ne doit pas lancer la production.
- Remplacez les `print` par `logging` : les traces restent exploitables quand le script tourne dans une chaîne.

```
import argparse, logging

def main():
    parser = argparse.ArgumentParser(description="Ingestion F1")
    parser.add_argument("entree")
    parser.add_argument("sortie")
    args = parser.parse_args()
    lignes = lire_resultats(args.entree)
    ecrire_courses_propres(args.sortie, lignes)
    logging.info("%d lignes ecrites vers %s", len(lignes), args.sortie)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    main()
```

Dernier détail qui compte quand la chaîne devient automatique : écrire le fichier de façon **atomique**. On écrit d'abord dans un fichier temporaire, puis on remplace : jamais de contrat à moitié écrit si le programme s'interrompt.

```
import os, tempfile

def ecrire_atomique(chemin, contenu):
    dossier = os.path.dirname(os.path.abspath(chemin))
    fd, tempo = tempfile.mkstemp(dir=dossier, suffix=".tmp")
    with os.fdopen(fd, "w", encoding="utf-8", newline="\n") as f:
        f.write(contenu)
    os.replace(tempo, chemin)    # bascule quasi instantanee
```

Exercices Python

- **2.a** Réécrivez `temps_en_secondes` en manipulant des millisecondes entières, puis prouvez que la sortie est identique sur les 50 lignes. Que gagne-t-on, que perd-on en lisibilité ?
- **2.b** Acceptez trois formats d'entrée : `1:33.614`, `1:33,614` et `93.614`. Refusez le reste avec un message qui cite la valeur fautive.
- **2.c** Faites afficher par la production un rapport : lignes lues, abandons, lignes rejetées, doublons supprimés.
- **2.d** Écrivez un test qui prouve l'idempotence du nettoyage : nettoyer le résultat d'un nettoyage ne change plus rien.
- **2.e** Calculez le SHA-256 de `courses_propres.csv` et affichez-le : c'est la façon la plus rapide de détecter une dérive de format entre deux exécutions.
- **2.f** Refusez d'écrire le fichier si une position sort de `0..20` ou si un nom de pilote est vide : un contrat faux vaut moins qu'un échec explicite.

Aide-mémoire Python

<code>texte.strip()</code>	# espaces + \n en trop
<code>texte.rstrip("\n")</code>	# seulement la fin de ligne
<code>texte.split(";")</code>	# vides finaux conservés
<code>texte.split()</code>	# découpe sur les espaces
<code>f"{valeur:.3f}"</code>	# 3 décimales garanties
<code>"" if v is None else f"{v:.3f}"</code>	# sentinelle -> champ vide
<code>math.isclose(a, b, abs_tol=1e-9)</code>	# comparer deux flottants
<code>open(p, encoding="utf-8-sig")</code>	# lecture (BOM tolère)
<code>open(p, "w", encoding="utf-8", newline="\n")</code>	# écriture LF
<code>unicodedata.normalize("NFD", t)</code>	# séparer lettres et accents
<code>enumerate(fichier, start=2)</code>	# numéro de ligne réel
<code>os.replace(temporaire, final)</code>	# écriture atomique

3. Java — un moteur typé, donc exigeant

Java n'accepte pas l'approximation : chaque champ a un type, et un type interdit certaines valeurs. La première décision du maillon est donc explicite — que vaut « rien » pour un temps, une position, une écurie ?

3.1 Le record : une ligne de données, immuable

```
public record Ligne(String course, String pilote, String ecurie,
                    int position, double tempsTour) { }
```

- record génère constructeur, equals, hashCode et toString : idéal pour une ligne de contrat, et impossible à modifier après création.
- Un double n'a pas de valeur « absente » : la convention du projet est -1. Cette convention doit être écrite en commentaire (c'est fait dans Ligne.java), sinon le prochain lecteur prend -1 pour un temps réel.
- Ne pas modifier les classes fournies n'est pas une politesse : c'est ce qui garantit que les tests fournis restent valides.

3.2 Le barème : un décalage d'indice, deux bornes à tester

```
public static final int[] BAREME = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };

public static int pointsPourPosition(int position) {
    if (position < 1 || position > BAREME.length) {
        return 0; // abandon (0) et au-dela du 10e
    }
    return BAREME[position - 1]; // 1re place -> indice 0
}
```

- Le - 1 est la source d'erreur numéro un des tableaux Java. Testez toujours les deux extrémités **et** leurs voisins : 1, 10, mais aussi 0 et 11.
- La garde position < 1 traite l'abandon sans code supplémentaire : un if (position == 0) séparé serait redondant.
- Si le barème évolue (points variables selon la course), un Map<Integer, Integer> avec getOrDefault(position, 0) remplace le tableau sans toucher aux appelants.

3.3 Agréger : de la boucle imbriquée à la table de hachage

```
// version lisible mais quadratique : pour chaque ligne, on relit la liste
for (Ligne ligne : lignes) {
    boolean trouve = false;
    for (Resultat r : resultats) {
        if (ligne.pilote().equals(r.nom)) { // recherche lineaire
            r.points += pointsPourPosition(ligne.position());
            trouve = true;
            break;
        }
    }
    if (!trouve) { ... }
}
```

```
// version qui passe à l'échelle : une seule lecture par ligne
Map<String, Resultat> parNom = new HashMap<>();
for (Ligne ligne : lignes) {
    Resultat r = parNom.computeIfAbsent(
        ligne.pilote(), nom -> new Resultat(nom, ligne.ecurie()));
    r.points += pointsPourPosition(ligne.position());
    if (ligne.position() == 1) r.victoires++;
    if (ligne.position() == 2) r.deuxiemes++;
}
List<Resultat> resultats = new ArrayList<>(parNom.values());
```

- `computeIfAbsent` remplace le trio « chercher / tester / insérer » : une seule lecture de la table, et l'initialisation est écrite à côté de la clé qui la déclenche.
- `HashMap` ne conserve aucun ordre. Ici ce n'est pas un problème puisque le tri suit ; s'il fallait un ordre d'insertion avant tri, ce serait `LinkedHashMap`.
- Coût : la version imbriquée est en $O(n \times p)$ (lignes $\times$ pilotes), la version avec table en $O(n)$. Sur 50 lignes l'écart est invisible ; sur 50 000 il décide si la page répond en une seconde.

Décision de modélisation à documenter

L'écurie d'un pilote est lue sur la première ligne rencontrée. Que faire s'il change d'écurie en cours de saison : garder la première, prendre la dernière, ou la plus fréquente ? Le code doit trancher explicitement, souvent par un commentaire de trois lignes — sinon deux lecteurs corrigeront le calcul dans deux directions opposées.

3.4 Compérateurs : un ordre total, sinon rien

```
Comparator<Resultat> ORDRE = Comparator
    .comparingInt((Resultat r) -> r.points).reversed()
    .thenComparing(r -> r.victoires, Comparator.reverseOrder())
    .thenComparing(r -> r.deuxiemes, Comparator.reverseOrder())
    .thenComparing(Resultat::nom);

resultats.sort(ORDRE);
```

- Les quatre critères de l'énoncé se lisent dans l'ordre : points décroissants, victoires, deuxièmes places, puis nom croissant.
- Piège de `reversed()` : appliqué à la fin de la chaîne, il inverse **tout** le compérateur, nom compris. En l'imbriquant par critère (ou avec `Comparator.reverseOrder()`), on garde le nom croissant.
- Ne jamais écrire `r2.points - r1.points` : la soustraction de deux `int` peut déborder (`Integer.MAX_VALUE - (-1)`), le compérateur devient incohérent et le tri lève `IllegalArgumentException: Comparison method violates its general contract!`
- Un compérateur doit être **total** : pour tout couple, exactement un résultat (négatif, zéro, positif), et transitif. Un compérateur qui compare « parfois avec la casse, parfois sans » peut produire $A < B$, $B < C$ et $C < A$: même exception, souvent seulement sur de gros tableaux.
- `List.sort` est stable, mais la stabilité ne doit jamais servir de critère caché : la comparaison finale sur le nom garantit un classement identique partout.
- `String.compareTo` compare des points de code Unicode : un nom accentué se place après « Z ». Ici les noms sont en majuscules sans accent, donc `compareTo` suffit ; pour un tri destiné à un humain, voir `Collator.getInstance(Locale.FRENCH)`.

3.5 Agréger les écuries sans corrompre les pilotes

```

public static List<Resultat> classementEcuries(List<Resultat> pilotes) {
    Map<String, Resultat> parEcurie = new HashMap<>();
    for (Resultat pilote : pilotes) {
        if (pilote.ecurie.isBlank()) {
            continue;
        }
        Resultat ecurie = parEcurie.computeIfAbsent(pilote.ecurie,
            nom -> new Resultat(nom, ""));    // objet NEUF
        ecurie.points += pilote.points;
        ecurie.victoires += pilote.victoires;
        ecurie.deuxiemes += pilote.deuxiemes;
    }
    List<Resultat> classement = new ArrayList<>(parEcurie.values());
    classement.sort(ORDRE);
    return classement;
}

```

- Le `new Resultat(nom, "")` est capital : réutiliser l'objet d'un pilote ferait grossir son total, et le bug n'apparaîtrait qu'au test suivant. Agréger, c'est créer.
- `Resultat` n'a pas de constructeur à trois arguments : les compteurs restent à zéro et on les additionne après. C'est voulu — le point d'entrée est unique.
- Se tromper de clé est l'erreur silencieuse de ce type de code : `pilote.nom` ici donnerait 10 « écuries », une par pilote, avec des totaux plausibles. Pour se protéger, ajouter une assertion de taille : `classement.size() == 5`.
- Bilan des compteurs à se poser avant d'écrire la boucle : quels champs s'additionnent (points, victoires, deuxièmes) et lesquels s'écrasent (nom, écurie) ?

3.6 positionMoyenne : arrondir, diviser, exclure

```

public static double positionMoyenne(List<Ligne> lignes, String pilote) {
    int somme = 0;
    int courses = 0;
    for (Ligne ligne : lignes) {
        if (!ligne.pilote().equals(pilote)) {    // equals, jamais ==
            continue;
        }
        if (ligne.position() == 0) {            // abandon exclu
            continue;
        }
        somme += ligne.position();
        courses++;
    }
    if (courses == 0) {
        return 0;                                // jamais classe
    }
    return Math.round((double) somme / courses * 100.0) / 100.0;
}

```

- `equals` et non `==` : `==` compare les références. Deux chaînes identiques peuvent être deux objets distincts, et le test « INCONNU » révèle vite cette erreur.
- `(double) somme / courses` : sans le cast, la division est **entière** et `7/4` vaut 1. Le test qui attend 1.5 échoue pour une raison qui n'a rien à voir avec les positions.
- `Math.round(x * 100) / 100.0` : attention, `Math.round(1.005 * 100)` renvoie 100, parce que 1.005 est stocké un cheveu en dessous. Pour un arrondi « commercial », utiliser `BigDecimal` et `RoundingMode.HALF_UP`.
- Renvoyer 0 quand le pilote n'a jamais terminé est une sentinelle acceptable **parce que** une position réelle commence à 1 : la valeur 0 ne peut pas être confondue avec une vraie moyenne. C'est le genre d'hypothèse qu'un commentaire doit rendre visible.

3.7 Lire le contrat : le second argument de split

```
String[] champs = ligne.split(";", -1);
double temps = champs[4].isBlank() ? -1 : Double.parseDouble(champs[4]);
```

- Le -1 est la limite du découpage, et il signifie « garder tous les champs, vides finaux compris ». Sans lui, la ligne `Bahrein;HAMILTON;Mercedes;0;` ne donne que 4 morceaux et `champs[4]` lève `ArrayIndexOutOfBoundsException`.
- C'est l'exact opposé de Python, qui conserve les vides finaux par défaut. Détail minuscule, bug très concret : l'abandon est justement la ligne qui finit par un champ vide.
- `Double.parseDouble` est indépendant de la locale : il attend le point décimal même sur un système français. En revanche `String.format("%.2f", x)` suit la locale et produit 1,50 en français : pour un format machine, écrire `String.format(Locale.ROOT, "%.2f", x)`.
- Détecter la sentinelle **avant** de parser : `parseDouble("")` lève une `NumberFormatException` dont le message ne dit ni quelle ligne, ni quelle course.
- `Files.newBufferedReader(Path.of(chemin), StandardCharsets.UTF_8)` : l'encodage explicite. Un vieux `FileReader` prend l'encodage de la machine et abîme les accents.

3.8 Exceptions : dire quoi, où, et pourquoi

```
try (BufferedReader lecteur = Files.newBufferedReader(
    Path.of(chemin), StandardCharsets.UTF_8)) {
    // ...
} catch (NumberFormatException cause) {
    throw new IllegalStateException(
        "ligne " + numero + " : temps illisible", cause);
}
```

- Une exception utile répond à trois questions : quelle entrée, quelle ligne, quelle règle. Le constructeur à deux arguments conserve la cause d'origine : `cause.getMessage()` seul perd la trace (stack trace) et rend le diagnostic deux fois plus long.
- `try (...)` ferme la ressource même en cas d'exception. Sous Windows, un fichier non fermé reste verrouillé et l'étape suivante du pipeline échoue à la lecture.
- Distinguer deux familles : **erreur de données** (attendue, on décide : tolérer, corriger, journaliser) et **bug** (inattendue, on laisse remonter et on corrige le code). Les confondre revient à cacher ses propres fautes derrière un « fichier invalide ».
- Renvoyer `null` pour signaler un échec est un anti-pattern : l'appelant oubliera de tester le cas, et l'erreur resurgira en `NullPointerException` trois classes plus loin. Un squelette Java qui écrit `return null;` est un piège à compléter, pas un contrat à conserver.

3.9 Écrire le contrat 2 sans casser le JavaScript

```
f.write("{\"nom\": \"" + r.nom + "\", \"points\": " + r.points + "}");
```

- Un nom contenant un guillemet ou un antislash produirait un fichier JS invalide. Deux échappements minimaux : `\"` puis `\\`.
- `donnees.js` n'est pas du JSON pur : il contient des déclarations `const`. Un analyseur JSON le refusera, et c'est voulu — ce fichier est exécuté par le navigateur, pas parsé.
- Un fichier généré est un artefact : on doit pouvoir le reproduire à l'identique. Si deux exécutions produisent deux fichiers différents, c'est qu'une source non déterministe s'est glissée dans le code (ordre d'une table de hachage, date, locale).

- Pour un échange de données sérieux, une bibliothèque JSON (Gson, Jackson) écrit et échappe correctement. Le choix « format lisible par tous contre dépendance en plus » revient en extension E4.

3.10 Extension E2 — le point du meilleur tour

```
public static String auteurMeilleurTour(List<Ligne> lignes, String course) {
    String auteur = null;
    double meilleur = Double.MAX_VALUE;
    for (Ligne ligne : lignes) {
        if (!ligne.course().equals(course)) continue;
        if (ligne.tempsTour() < 0) continue; // aucun temps releve
        if (ligne.tempsTour() < meilleur) {
            meilleur = ligne.tempsTour();
            auteur = ligne.pilote();
        }
    }
    return auteur; // null si aucun temps
}
```

- **Deux règles distinctes, et c'est tout le piège** : le meilleur tour se calcule sur tous les temps relevés, abandons compris ; le point n'est attribué que si l'auteur du tour termine dans les dix premiers.
- Le jeu de test de E2 place justement un abandon en tête du classement du meilleur tour : un code qui fusionne les deux règles donne un point à personne, ou à quelqu'un qui n'a pas terminé.
- Initialiser avec `Double.MAX_VALUE` et non 0 : avec 0, aucun temps ne serait jamais « plus petit » et la fonction renverrait toujours null.
- Égalité de temps : < garde le premier rencontré, <= le dernier. L'énoncé ne précise pas, donc le choix se documente en commentaire. Ne comptez pas sur l'ordre des lignes du fichier pour trancher.
- `classementAvecMeilleurTour` réutilise `classementPilotes` puis ajoute 1 point aux auteurs concernés : dupliquer le calcul complet serait le meilleur moyen de faire diverger les deux classements.
- Attention à la définition de « classé dans les dix » : c'est la position dans la course (le champ `position`), pas le rang au championnat. Relire l'énoncé plutôt que supposer.

Exercices Java

- **3.a** Remplacez la boucle imbriquée par une `HashMap` et vérifiez que les cinq tests restent au vert. Écrivez les deux versions côte à côte et expliquez à voix haute pourquoi elles donnent le même résultat.
- **3.b** Ajoutez un comparateur « position moyenne croissante, puis nom » en réutilisant une seule fois le calcul de moyenne (le résultat est coûteux : ne le recalculez pas dans la comparaison, un comparateur peut être appelé des dizaines de milliers de fois).
- **3.c** Fabriquez deux `Resultat` avec des points proches de `Integer.MAX_VALUE`, trie-les avec un comparateur soustractif, et observez `IllegalArgumentException`. Puis corrigez avec `Integer.compare`.
- **3.d** Ajoutez dans `classementEcuries` l'assertion `classement.size() == 5`, puis utilisez volontairement `pilote.nom` comme clé : constatez le message.
- **3.e** Retirez le `(double)` de `positionMoyenne`, lancez les tests, puis expliquez dans un commentaire pourquoi 1.5 devient 1.
- **3.f** Faites échouer volontairement la lecture de `courses_propres.csv` (renommez-le) et vérifiez que le message du test 5 dit bien quel maillon est en cause.
- **3.g** Générez un mini-contrat 2 à partir d'un `Resultat` dont le nom contient un guillemet, puis chargez le fichier dans le navigateur : que se passe-t-il avant et après l'échappement ?

Aide-mémoire Java

```

ligne.split(";", -1)           // garde les champs vides finaux
s.isBlank()                   // vide ou uniquement des espaces
s.equals(autre)               // comparer des chaines, jamais ==
Integer.compare(a, b)         // comparateur sans debordement
Comparator.comparingInt(Resultat::points).reversed()
    .thenComparing(Resultat::nom)
map.computeIfAbsent(cle, k -> new Resultat(k, ""))
map.values()                  // collection des valeurs
new ArrayList<>(map.values())  // copie triable
(double) somme / courses      // division reelle, pas entiere
Math.round(x * 100) / 100.0   // arrondi a deux decimales
Double.parseDouble("93.614")  // point decimal, locale libre
String.format(Locale.ROOT, "%.2f", x)
Files.newBufferedReader(Path.of(p), StandardCharsets.UTF_8)
try (BufferedReader r = ...) { } // fermeture automatique
throw new IllegalStateException(msg, cause)

```

4. JavaScript — trier, afficher, ne rien casser

Dernier maillon, et le plus exposé : le code s'exécute dans un navigateur, où l'erreur ne se voit qu'à l'exécution, et où chaque écriture dans la page modifie un état partagé — le DOM.

4.1 `sort()` : trois pièges dans une seule fonction

```
[9, 25, 100].sort()           // [100, 25, 9] : compare du TEXTE
```

```
const trie = [...liste].sort((a, b) =>  
  b.points - a.points || b.victoires - a.victoires);
```

- Sans comparateur, `sort()` convertit les éléments en chaînes : `100` passe avant `25`. Sur des nombres, un comparateur est obligatoire.
- `sort()` trie **sur place** : d'où la copie `[...liste]`. Une copie **superficielle** suffit ici parce que la fonction ne modifie aucun objet — elle se contente de réordonner les références. `structuredClone` serait inutile et coûteux.
- Un comparateur doit renvoyer un nombre négatif, nul ou positif. Renvoyer un booléen « marche » parfois, mais deux éléments peuvent alors être tous les deux « inférieurs » : le tri devient incohérent.
- `a || b` enchaîne les critères : le second n'est évalué que si le premier vaut `0` (`0` est *falsy*). Compact, mais à condition de comprendre le raccourci.
- `sort` est **stable** depuis ES2019 : deux éléments égaux gardent leur ordre relatif. Ne pas bâtir une règle métier là-dessus : ajoutez un critère final (le nom) pour un ordre identique sur tous les navigateurs.
- Pour un tri alphabétique correct avec accents : `a.nom.localeCompare(b.nom, "fr")`. Et pour tester un nombre non calculé : `Number.isNaN(x)`, pas `isNaN("abc")` qui convertit d'abord en nombre.

4.2 Copier, muter, partager

```
const copie = [...PILOTES];      // tableau neuf, memes objets  
Object.freeze(PILOTES);         // gel superficiel, en developpement  
  
// renvoient un tableau neuf  
liste.filter(...); liste.map(...); liste.slice(); [...liste];  
  
// modifient la liste recue  
liste.sort(...); liste.push(...); liste.splice(...); liste.reverse();
```

- Reconnaître les deux familles de méthodes évite la moitié des bugs de ce maillon.
- « La liste reçue ne doit pas être modifiée » n'est pas une coquetterie : si plusieurs fonctions trient tour à tour la même liste partagée, l'affichage dépend de l'ordre des appels, et le bug apparaît seulement quand on clique sur une colonne.
- `Object.freeze` est superficiel : il gèle le tableau, pas les objets qu'il contient. Pour un vrai gel, il faut aussi geler chaque objet.

4.3 Le DOM n'est pas un tableau : coûts et pièges

```

const corps = document.getElementById(idCorps);
corps.replaceChildren(); // vide sans reparser du HTML

for (let i = 0; i < liste.length; i++) {
  const ligne = document.createElement("tr");
  ligne.dataset.nom = liste[i].nom; // attribut data-nom

  const cellules = ["", "", "", "", ""] // 5 emplacements reels
    .map(() => document.createElement("td"));

  cellules[0].textContent = i + 1;
  cellules[1].textContent = liste[i].nom;
  cellules[2].textContent = liste[i].ecurie || "";
  cellules[3].textContent = liste[i].points;
  cellules[4].textContent = liste[i].victoires;

  ligne.append(...cellules); // 5 cellules, 1 operation
  corps.appendChild(ligne); // puis 1 insertion dans la page
}

```

- **Piège du tableau creux** : `Array(5).map(f)` ne produit rien, parce que `map` saute les cases vides d'un tableau creux. La liste de cinq chaînes utilisée ici est un moyen simple (et explicite) de contourner le problème.
- `replaceChildren()` vide le conteneur d'un coup. `innerHTML = ""` y parvient aussi mais refait passer la chaîne par l'analyseur HTML — inutile ici.
- `innerHTML +=` dans une boucle est le pire choix possible : tout le sous-arbre est sérialisé puis reconstruit, les écouteurs disparaissent, et un nom de pilote contenant du balisage est interprété. C'est une injection (XSS).
- `textContent` écrit du texte brut, jamais du HTML, et convertit les nombres en chaînes : `cellules[3].textContent = 101` puis `cellules[3].textContent === "101"`. En lecture, une cellule renvoie **toujours** une chaîne.
- Les cellules sont créées puis assemblées alors que la ligne n'est pas encore dans le document : une seule insertion visible. Insérer chaque cellule dans la page vivante multiplie les recalculs de mise en page (*reflow*).
- `append` accepte plusieurs nœuds (et du texte), `appendChild` un seul nœud et renvoie l'enfant ajouté. Utiliser les deux dans le même bloc n'est pas une incohérence, c'est l'API telle qu'elle est faite.

4.4 Événements : pourquoi l'écouteur doit vivre ailleurs

```

// fragile : la ligne est recreee a chaque affichage,
// donc l'ecouteur pose ici disparaît au rendu suivant
ligne.addEventListener("click", ...);

// robuste : un seul ecouteur sur le conteneur stable
tableau.addEventListener("click", (evenement) => {
  const entete = evenement.target.closest("th[data-colonne]");
  if (entete === null) return; // clic hors d'un en-tete
  trierEtAfficher(entete.dataset.colonne);
});

```

- Trois stratégies pour un élément recréé : recréer l'écouteur à chaque rendu, **déléguer** à un ancêtre stable, ou ne pas recréer les lignes. La délégation est presque toujours la moins coûteuse à maintenir.
- La délégation repose sur la propagation (*bubbling*) : le clic remonte du plus profond jusqu'au conteneur, qui reçoit l'événement pour tous ses descendants.
- `evenement.target` est l'élément réellement cliqué, pas celui qui écoute : il peut être un texte, un `<span>`, un `<td>`. `closest()` remonte l'arbre jusqu'à trouver l'ancêtre qui correspond.

- `removeEventListener` exige la **même référence** de fonction : un écouteur anonyme ne peut pas être retiré. Pour un usage unique, `{ once: true }` est plus simple et plus lisible que du nettoyage manuel.

4.5 L'état d'affichage : les données sont la seule vérité

```
// etat = ce que l'utilisateur a demande (jamais deduit du DOM)
const etat = { ecurie: "", colonne: "points" };

function rendre() {
  const filtre = etat.ecurie === ""
    ? PILOTES
    : PILOTES.filter((p) => p.ecurie === etat.ecurie);
  const liste = trierSelon(filtre, etat.colonne);
  remplirTableau("corps-pilotes", liste);
  marquerPodium("corps-pilotes");
}
```

- Le DOM est un **rendu jetable**, pas une base de données. Déduire l'état en relisant la page (« la première ligne a 101 points, donc le tableau est trié par points ») est lent et faux dès qu'un filtre s'applique.
- Un seul point d'entrée `rendre()` après chaque changement d'état : sinon on obtient un podium marqué sur un tableau obsolète, ou un filtre appliqué à un classement déjà réordonné.
- Chaîne de traitement : **filtrer** → **trier** → **remplir** → **décorer**. Toutes les étapes sauf la dernière travaillent sur des tableaux et n'écrivent rien dans la page : c'est ce qui les rend testables sans navigateur.
- Rassembler l'état dans un objet unique plutôt que dans cinq variables globales : on peut alors le comparer, le journaliser, le restaurer (« reviens à la vue par défaut »).
- `trierParPoints(PILOTES)` renvoie une copie : `PILOTES` garde l'ordre du fichier généré. C'est précisément ce qui permet d'enchaîner deux tris différents sans que le second dépende du premier.

4.6 Tester dans le navigateur, sans se raconter d'histoires

- Les tests fournis manipulent `document` : ils ne peuvent donc pas tourner dans une simple console Node sans un DOM simulé (`jsdom`).
- Ouvrir `index.html` par double-clic (`file://`) suffit ici. Le jour où vous ajoutez `fetch` ou des modules ES, il faut un petit serveur local : `python -m http.server 8000`.
- La console (F12) affiche les mêmes messages que la page, avec le détail « attendu / obtenu » : c'est le premier endroit à regarder, avant de modifier quoi que ce soit.
- `node --check app.js` vérifie la syntaxe sans exécuter le fichier. C'est un filet minimum contre les fautes de frappe, pas un test : aucune garantie sur le comportement.
- Pour aller plus loin sans navigateur : `node --test` avec `jsdom` pour le DOM, ou `Playwright` si vous voulez vraiment cliquer. À mettre en balance avec le temps d'installation.
- **Astuce de diagnostic** : testez vos fonctions une par une dans la console du navigateur (`trierParPoints([{nom: 'A', points: 1, victoires: 0}])`) plutôt que de relancer la page entière pour chaque essai.

4.7 Extension E3 — filtrer et trier sans casser l'affichage

```
function filtrerParEcurie(liste, ecurie) {
  if (ecurie === "") {
    return [...liste]; // copie : l'original reste intact
  }
  return liste.filter((entree) => entree.ecurie === ecurie);
}

const COMPAREURS = {
  nom: (a, b) => a.nom.localeCompare(b.nom, "fr"),
  points: (a, b) => b.points - a.points,
  victoires: (a, b) => b.victoires - a.victoires,
};

function trierSelon(liste, colonne) {
  const compareur = COMPAREURS[colonne] || COMPAREURS.points;
  return [...liste].sort(compareur);
}
```

- Renvoyer une copie même quand aucun filtre n'est appliqué : l'appelant ne peut alors pas modifier la source par accident.
- Une table de comparateurs remplace la cascade de `if` et rend chaque colonne indépendante : ajouter une colonne = ajouter une ligne.
- Le tri doit s'appliquer **après** le filtre, et la colonne choisie doit être mémorisée dans l'état : sinon le clic suivant repart de la liste complète et la vue revient à son ordre initial.
- Le podium se décore **en dernier**, après le nouveau remplissage : il suit l'ordre affiché, pas l'ordre logique des données.

Exercices JavaScript

- **4.a** Écrivez `trierPar(liste, cle, ordre)` qui trie selon une clé et un sens, sans jamais modifier l'entrée. Prouvez-le par un test qui compare la liste avant/après appel.
- **4.b** Remplacez la liste de cinq chaînes par `Array.from({length: 5}, () => document.createElement("td"))`, puis expliquez dans un commentaire pourquoi `Array(5).map(...)` ne produit rien.
- **4.c** Affichez 10 000 lignes d'abord avec la méthode actuelle, ensuite avec `innerHTML +=`, en mesurant avec `performance.now()`. Notez le rapport entre les deux durées.
- **4.d** Construisez le contenu dans un `DocumentFragment` et n'insérez qu'une fois : mesurez à nouveau, puis expliquez l'écart par le nombre de recalculs de mise en page.
- **4.e** Ajoutez un compteur d'appels à `remplirTableau` : un clic sur une colonne doit provoquer exactement un remplissage, pas deux.
- **4.f** Posez la délégation d'événement sur le `<table>` au lieu des `<th>`. Vérifiez que le tri fonctionne encore après trois réaffichages, puis supprimez la délégation pour constater la panne.
- **4.g** Injectez un nom de pilote valant `<img src=x onerror=alert(1)>` via `textContent`, puis via `innerHTML`. Consignez ce qui s'exécute et pourquoi.

Aide-mémoire JavaScript

```
[...liste].sort((a, b) => b.points - a.points) // copie + comparateur
liste.filter((x) => x.ecurie === e)           // tableau neuf
Array.from({length: n}, (_, i) => ...)         // pas de trous
corps.replaceChildren();                      // vider proprement
element.textContent = valeur;                 // du texte, jamais du HTML
element.dataset.nom = "VERSTAPPEN";           // attribut data-nom
ligne.append(...cellules);                    // plusieurs noeuds d'un coup
evenement.target.closest("th[data-colonne]")  // delegation
Number.isNaN(x); typeof x === "number";       // tester proprement
a.nom.localeCompare(b.nom, "fr");             // tri alphabetique lisible
structuredClone(objet);                      // copie profonde
performance.now();                           // mesurer, pas supposer
```

5. Méthode — déboguer, vérifier, durcir

5.1 Un protocole de débogage en six étapes

- **1. Lire le message en entier.** Fichier, ligne, type d'exception. Une erreur bien lue est à moitié résolue.
- **2. Reproduire en plus petit.** Deux lignes de données et un appel suffisent presque toujours à reproduire un bug. Si vous n'y arrivez pas, c'est que le cas n'est pas encore compris.
- **3. Situer la couche.** Le bug est-il dans la lecture, dans la conversion, dans le calcul ou dans l'affichage ? Chaque couche a son fichier et ses tests.
- **4. Formuler une hypothèse, puis la tester.** Un seul affichage ciblé au bon endroit vaut mieux que dix affichages partout. Écrivez ce que vous attendez **avant** de lancer le programme.
- **5. Corriger la cause.** Un `if` ajouté pour contourner le symptôme garantit que le bug reviendra ailleurs.
- **6. Rejouer.** Les tests du maillon, puis la chaîne complète. Une correction qui casse la chaîne n'est pas une correction.

Le symptôme est rarement là où on le voit

Un maillon Java à 2/5 peut venir d'un fichier Python jamais produit. Un classement faux côté page web peut venir d'un champ mal échappé côté Java. Avant de corriger, identifiez **quel fichier** contient l'information fautive — c'est presque toujours celui d'avant.

5.2 Diagnostic rapide : symptôme, cause probable, vérification

Symptôme	Cause probable	Vérification
Test 5 : « courses_propres.csv illisible »	Le maillon Python n'a pas été exécuté, ou le chemin est relatif	Lister le dossier depuis lequel java est lancé
Tous les pilotes à 0 point	Barème mal indexé ou position mal lue	Appeler <code>pointsPourPosition</code> pour 1, 10, 11 et 0
Un pilote présent deux fois	Clé d'agrégation incohérente (nom ou casse)	Afficher les clés distinctes avant d'agréger
Classement différent à chaque exécution	Comparateur incomplet, ou tri d'une table de hachage	Trier deux fois et comparer, ajouter le critère final
Position moyenne étrange	Abandons comptés, ou division entière	Afficher la somme et le nombre de courses prises
Page blanche, compteurs figés	Erreur JavaScript à l'exécution	Ouvrir la console et lire la première erreur
Le podium ne suit pas le classement	Décoration faite avant le remplissage	Vérifier l'ordre des appels dans le rendu
Accents cassés dans un fichier	Lecture ou écriture sans encodage explicite	Ouvrir le fichier en binaire et vérifier l'UTF-8
Fin de ligne refusée par la comparaison	CRLF au lieu de LF (<code>csv.writer</code> , éditeur)	Comparer les premiers octets avec la référence
Le mot None apparaît dans le CSV	Sentinelle non convertie à l'écriture	Rechercher la chaîne None dans le fichier produit

5.3 Ce que veut dire « maillon terminé »

- Les tests du maillon passent **relancés depuis zéro**, pas seulement après une suite de modifications à chaud.
- La chaîne complète est rejouée depuis le CSV brut et produit le résultat attendu.

- Aucune valeur sentinelle ne subsiste dans les fichiers produits (ni None, ni -1, ni champ vide inattendu).
- Le fichier généré est reproductible : deux exécutions donnent deux fois le même contenu, octet pour octet.
- Le README dit la vérité, y compris sur ce qui ne marche pas encore. Situer précisément son avancement est une compétence, pas un aveu.

5.4 Git : un historique qu'on peut relire

- Un commit = une intention. « Corriger le barème et le README » est deux commits, pas un.
- Message à l'impératif présent (« Corriger », « Ajouter », « Extraire ») ; il décrit le **pourquoi** quand le diff montre déjà le **quoi**.
- `git diff` avant chaque `git add` : c'est la relecture la moins chère du projet.
- Ne jamais committer les artefacts : `02-java/out/`, `.venv/`, `__pycache__`/, `.ipynb_checkpoints/`, les fichiers temporaires et les journaux.
- Une branche par extension, fusion quand les tests passent. Sur un dépôt partagé, ne réécrivez jamais l'historique publié : `git revert` est plus honnête que `--force`.

5.5 Checklist de revue, avant de dire « c'est fini »

- Le contrat de sortie est identique à la référence : colonnes, séparateurs, décimales, fin de ligne.
- Aucune sentinelle ne traverse une frontière de langage sans conversion.
- Les classes et fichiers fournis n'ont pas été modifiés.
- Les tests passent dans un environnement neuf, sans fichier laissé par une exécution précédente.
- Les trois maillons s'enchaînent en repartant du fichier brut.
- Les échecs restants sont documentés dans le README, avec le nombre de tests réel.
- Aucun temporaire, journal ou binaire de compilation n'est suivi par Git.
- Les noms de variables disent ce qu'elles contiennent (`points`, pas `total2`).
- Les commentaires expliquent les décisions, pas la syntaxe : « sentinelle -1 = pas de temps » est utile, « on boucle sur la liste » ne l'est pas.
- Un lecteur extérieur comprend le README en trois minutes et sait exactement où reprendre le travail.

6. Les extensions, une par une

Les extensions ne sont pas des exercices facultatifs : elles font apparaître les problèmes que le socle évite volontairement (données sales, règle conditionnelle, interface qui vit, automatisation). Chacune a sa place dans un entretien technique.

6.1 E1 — les données sales (Python)

- **Objectif** : produire le contrat 1 à partir de l'export brut réel, sans changer une seule ligne du reste.
- **Ce qui est vraiment évalué** : la capacité à écrire un pipeline de normalisation **ordonné** (espaces, casse, accents, écuries, séparateur décimal, validation, doublons) et à décider ce qu'on tolère.
- **Pièges** : `.title()` qui donne McLaren ; un replace à la main qui oublie un accent ; un set qui détruit l'ordre du contrat ; une ligne sans pilote qui devient une fausse écurie.
- **Réussi quand** : votre sortie est identique ligne à ligne à `reference/courses_propres.csv`, et votre code affiche combien de lignes ont été rejetées et pourquoi.

6.2 E2 — le point du meilleur tour (Java)

- **Objectif** : ajouter une règle conditionnelle sans dupliquer le calcul du classement.
- **Ce qui est vraiment évalué** : séparer deux règles qui se ressemblent — déterminer l'auteur du meilleur tour (abandons compris) et attribuer le point (seulement si l'auteur est classé dans les dix).
- **Pièges** : initialiser le minimum à 0 ; oublier la sentinelle -1 qui signifie « aucun temps » ; compter le point deux fois si l'auteur est aussi premier ; confondre position en course et rang au championnat.
- **Réussi quand** : `TestsE2` passe, et `donnees.js` régénéré avec le nouveau classement reste chargé par la page sans erreur.

6.3 E3 — une interface vivante (JavaScript)

- **Objectif** : filtrer par écurie et trier au clic, sans jamais corrompre les données reçues.
- **Ce qui est vraiment évalué** : la distinction entre **source**, **état d'affichage** et **rendu**, plus la délégation d'événements.
- **Pièges** : `sort()` sur la liste d'origine ; un comparateur alphabétique sur des nombres ; un écouteur posé sur des `<tr>` recréés ; le podium décoré avant le réaffichage ; un filtre qui oublie le tri courant.
- **Réussi quand** : cliquer trois fois de suite sur les mêmes en-têtes laisse l'affichage cohérent, filtre et tri compris, et `PILOTES` n'a pas bougé (testez avec `Object.freeze(PILOTES)`).

6.4 E4 — la chaîne complète

- **Objectif** : une commande qui rejoue l'ingestion, la compilation, l'exécution et le contrôle.
- **Ce qui est vraiment évalué** : le traitement des codes de retour et des chemins relatifs, et le fait de s'arrêter au premier maillon fautif avec un message clair.
- **Pièges** : lancer un subprocess sans vérifier son code de retour ; le répertoire courant qui n'est pas celui du script ; un stdout d'encodage différent sous Windows ; des chemins en dur propres à une seule machine.
- **Pour aller plus loin** : remplacer le pivot CSV par du JSON. Côté Python `json.dump` suffit ; côté Java il faut écrire un mini-analyseur ou ajouter une bibliothèque (Gson, Jackson) en `.jar` local. C'est exactement l'arbitrage d'un projet réel : un format lisible par tous contre une dépendance de plus.

6.5 Aller au-delà du pack

- **Validation par schéma** : décrire le contrat (colonnes, types, bornes) et le vérifier automatiquement avant chaque étape. Côté Python, une `dataclass` avec `__post_init__` ou un modèle Pydantic fait le travail.

- **Tests hors du pack** : pytest pour Python, JUnit pour Java, node --test + jsdom pour le JavaScript. Les tests fournis restent la référence ; les vôtres couvrent les cas limites que le pack ne teste pas.
- **Intégration continue** : une action qui exécute l'ingestion, les tests Java et la vérification de la chaîne à chaque push. Le jour où vous cassez un contrat, vous le savez en trois minutes au lieu de trois jours.
- **Performance** : générer un million de lignes et chronométrer chaque maillon. Cela transforme les remarques de complexité du chapitre 3 en mesures — la seule façon défendable de parler de performance.
- **Base de données** : à partir de plusieurs saisons, remplacer les fichiers par SQLite et une requête GROUP BY pilote, ecurie. Le calcul des points reste le même : c'est le signe que la logique était bien séparée des données.

7. Aide-mémoire : la même opération, trois langages

Besoin	Python	Java	JavaScript
Parcourir une collection	for ligne in lignes:	for (Ligne l : lignes)	for (const l of liste)
Regrouper par clé	dico.setdefault(cle, []).append(x)	map.computeIfAbsent(cle, k -> new Resultat(k))	liste.reduce((m, x) => ..., new Map())
Trier avec critères	sorted(l, key=lambda x: (-x.points, x.nom))	Comparator.comparingInt(...).thenComparing(...)	[...l].sort((a, b) => b.points - a.points)
Comparer deux textes	a == b	a.equals(b)	a === b
Texte vide ou espaces	not texte.strip()	texte.isBlank()	texte.trim() === ""
Arrondir à 2 décimales	round(x, 2)	Math.round(x * 100) / 100.0	x.toFixed(2) ← renvoie une chaîne
Minimum d'une liste	min(valeurs)	valeurs.stream().min(...)	valeurs.reduce((a, b) => Math.min(a, b))
Rang affiché	enumerate(l, start=1)	i + 1	i + 1
Signaler une donnée invalide	raise ValueError("ligne 37")	throw new IllegalArgumentException(...)	throw new Error(...)
Lire un fichier UTF-8	open(p, encoding="utf-8-sig")	Files.newBufferedReader(Path.of(p), UTF_8)	await (await fetch(p)).text()
Écrire un fichier en LF	open(p, "w", encoding="utf-8", newline="\n")	Files.newBufferedWriter(Path.of(p), UTF_8)	impossible dans le navigateur

Les lignes de ce tableau se ressemblent, mais trois d'entre elles cachent un piège réel :

- `toFixed(2)` renvoie une **chaîne**, pas un nombre : inutilisable pour continuer un calcul.
- `Math.min(...valeurs)` passe chaque élément en argument ; au-delà de quelques dizaines de milliers d'éléments, la pile d'appels déborde. Une boucle ou `reduce` est plus sûre.
- En Python, `sorted()` renvoie une copie alors que `liste.sort()` trie sur place ; en Java c'est l'inverse selon que vous utilisez `List.sort` ou un flux ; en JavaScript, `sort` modifie toujours. Trois langages, trois conventions : d'où l'intérêt de toujours écrire la copie explicitement.

Les six réflexes à garder

- Écrire ce qu'on attend **avant** de lancer le programme.
- Convertir une sentinelle à la frontière, jamais au milieu.
- Tester les bords : 0, 1, la dernière valeur, la valeur absente.
- Nommer les variables comme leurs voisins les nomment.
- Commenter les décisions, pas la syntaxe.
- Rejouer toute la chaîne après chaque correction.

8. Feuille de route : ce qui reste, et dans quel ordre

État réel du dépôt : le maillon Python est au vert (4/4), le maillon Java est à 2/5, le maillon JavaScript est écrit mais ses tests n'ont pas encore été passés dans le navigateur. Deux fonctions Java manquent : `classementEcuries` et `positionMoyenne`.

8.1 Les six étapes, dans l'ordre

- [] **1. Écrire `classementEcuries`.** Clé = nom d'écurie, `Resultat` neufs (copie défensive), addition des points, victoires et deuxièmes places, puis tri avec le même comparateur. Résultat attendu : 3/5.
- [] **2. Écrire `positionMoyenne`.** Ignorer les positions 0, additionner et compter les courses terminées, caster en double avant la division, arrondir à deux décimales, renvoyer 0 si aucune course terminée. Résultat attendu : 5/5.
- [] **3. Produire puis vérifier la page.** Lancer `Main` pour régénérer `donnees.js`, ouvrir `index.html` et lire les tests : le maillon JavaScript doit afficher 5/5, puis le classement complet à l'écran.
- [] **4. Figer le socle.** Copier tout le pack dans `socle_valide/` (ou créer un tag Git) **avant** d'attaquer les extensions : c'est votre point de retour si une extension tourne mal.
- [] **5. Enchaîner E1 → E2 → E3 → E4**, une branche par extension, fusion seulement quand les tests de l'extension passent et que le socle reste intact.
- [] **6. Dernière passe sur le README** : état exact, ce qui marche, ce qui ne marche pas, et le score réel de chaque maillon.

8.2 Comment savoir que vous progressez vraiment

Compétence	Preuve concrète	Exercice
Sentinelles et contrats	Vous expliquez sans hésiter la différence entre 0, None, "" et -1, et où chacune vit	1.1, 2.4
Parsing défensif	Votre convertisseur accepte trois formats et refuse le reste en citant la valeur fautive	2.b
Nombres flottants	Vous justifiez <code>round(x, 3) + f"{x:.3f}"</code> et l'alternative en millisecondes entières	2.a, 2.2
Complexité	Vous passez d'une boucle imbriquée à une table de hachage en expliquant le coût des deux	3.a, 6.5
Comparateurs	Vous écrivez un ordre total en quatre critères et vous savez pourquoi une soustraction peut lever une exception	3.b, 3.c
DOM	Vous choisissez <code>textContent</code> , vous expliquez l'injection par <code>innerHTML</code> et le coût d'un reflow	4.3, 4.c, 4.g
Événements	Vous déléguez au conteneur stable et vous expliquez la propagation	4.4, 4.f
Méthode	Vous réduisez un bug à deux lignes de données avant de toucher au code	5.1
Git	Votre historique se relit <code>commit</code> par <code>commit</code> , sans artefact suivi	5.4

La seule conclusion qui compte

Un projet n'est pas terminé quand les tests passent : il est terminé quand vous pouvez expliquer chaque ligne à quelqu'un qui ne l'a pas écrite — et dire précisément ce que vous n'avez pas encore réussi.

Ce guide ne remplace pas le travail ; il sert à poser les bonnes questions au bon moment. Reprenez-le chapitre par chapitre au fur et à mesure des maillons, et refaites les exercices dans le vrai code : c'est là que la progression se mesure.